

Migración de Arquitectura Industrial: Java RMI a API REST

Documento de Especificación Técnica e Ingeniería de Software — Framework FastAPI

1. Transformación de Métodos a Recursos REST

La migración desde un paradigma de Objetos Distribuidos (Java RMI) hacia una arquitectura orientada a recursos (REST) implica un cambio fundamental en el diseño de las comunicaciones. En Java RMI, el cliente interactúa de forma directa mediante la invocación de firmas lógicas de métodos expuestos en una interfaz remota (`IServidorIndustrial`). En el nuevo modelo implementado con Python y FastAPI, las acciones se desacoplan por completo y pasan a mapearse de forma semántica combinando los métodos estándar del protocolo HTTP con URIs jerárquicas que representan los recursos físicos y lógicos de la planta industrial (como sensores, umbrales y alertas).

A continuación, se detalla la tabla de equivalencias exacta que estructura la transformación completa de los servicios del servidor industrial:

Método Java RMI Original	Método HTTP	Estructura del Recurso (URI)	Cuerpo JSON / Parámetros
<code>enviarMedicion(...)</code>	POST	<code>/mediciones</code>	Body: <code>{idSensor, timestamp, variable, valor, unidad}</code>
<code>consultarHistoricoMediciones(...)</code>	GET	<code>/mediciones/{id_sensor}/{variable}</code>	Query string: <code>? inicio=X&fin=Y</code> (opcionales, 0 por defecto)
<code>consultarUmbrales(...)</code>	GET	<code>/umbrales/{id_sensor}/{variable}</code>	Ninguno. Devuelve matriz de límites <code>[min, max]</code> .
<code>consultarHistoricoAlertas(...)</code>	GET	<code>/alertas/{id_sensor}/{variable}</code>	Query string: <code>? inicio=X&fin=Y</code> (opcionales)
<code>configurarUmbrales(...)</code>	PUT	<code>/umbrales/{id_sensor}/{variable}</code>	Body: <code>{min, max}</code> (Sobrescribe umbral específico)

2. Diferencias respecto a Objetos Distribuidos

El modelo de objetos distribuidos (RMI) y la arquitectura REST (Representational State Transfer) pertenecen a filosofías conceptuales drásticamente distintas:

- **Nivel de Acoplamiento:** Java RMI requiere un acoplamiento binario y de plataforma sumamente estricto. Tanto el cliente como el servidor deben ejecutarse en la Máquina Virtual de Java (JVM) y compartir los archivos de las clases (o stubs compartidos). En cambio, la API REST en Python se apoya en un desacoplamiento total: la comunicación es puramente textual (JSON sobre HTTP), lo que permite que un cliente desarrollado en cualquier tecnología interactúe con el servidor sin dependencias compartidas de compilación.
- **Abstracción de la Red:** RMI busca la "transparencia de localización", intentando engañar al programador al hacer que una llamada remota parezca una llamada local en memoria. Esto suele ocultar los problemas reales de latencia y fallos de red. REST, por su parte, expone la red de forma explícita y nativa, utilizando códigos de estado HTTP estandarizados (como `200 OK` o `404 Not Found`) para reflejar con precisión el resultado de las interacciones.
- **Protocolo de Transporte:** Mientras que RMI utiliza un protocolo propietario de Java (JRMP) que requiere mantener conexiones TCP persistentes y complejas negociaciones de puertos dinámicos altos, REST funciona sobre HTTP/HTTPS estándar, facilitando enormemente el tránsito a través de firewalls y proxies industriales sin configuraciones complejas de infraestructura.

3. Ventajas e Inconvenientes del Modelo REST

Ventajas Principales:

- **Interoperabilidad Universal:** Al utilizar JSON como formato de intercambio, cualquier elemento de la planta industrial (un script en Python, una aplicación móvil, un panel HMI web en JavaScript o un PLC con soporte HTTP) puede consumir los datos directamente sin stubs propietarios.
- **Escalabilidad y Flexibilidad:** Al no depender de hilos bloqueantes persistentes asociados a una sesión de objeto vivo, el servidor FastAPI puede manejar un volumen sustancialmente mayor de peticiones concurrentes a través de controladores asíncronos nativos (`async/await`).
- **Inspección y Pruebas Sencillas:** Las herramientas de diagnóstico se simplifican radicalmente. La propia documentación automática interactiva (Swagger UI en `/docs`) o clientes HTTP como Bruno permiten validar el estado completo de la planta enviando textos simples, lo que reduce drásticamente los tiempos de desarrollo y depuración.

Inconvenientes:

- **Mayor consumo de ancho de banda:** La serialización y transferencia de texto en formato JSON junto con las cabeceras HTTP de cada petición genera un consumo de bytes significativamente mayor que el formato binario nativo empaquetado por JRMP en RMI.
- **Ausencia de tipado estricto en la red:** RMI valida las interfaces y tipos de datos en tiempo de compilación. En REST, si bien herramientas como `Pydantic` en FastAPI validan fuertemente el JSON al recibirlo, los errores de formato o tipos incorrectos se detectan exclusivamente en tiempo de ejecución,

obligando al servidor a validar los cuerpos de los mensajes manualmente y a responder con códigos `422 Unprocessable Entity`.

4. Discusión Técnica y Problemas Abordados

A. El Paradigma Stateless (Sin Estado)

Uno de los pilares de REST es la restricción de que cada petición HTTP debe ser completamente autocontenida. El servidor no mantiene ningún contexto de sesión ni guarda constancia de qué cliente está realizando la llamada. En nuestro sistema industrial, esto significa que el endpoint `GET /mediciones/{id_sensor}/{variable}` procesa la solicitud de forma inmediata basándose únicamente en los parámetros recibidos en la ruta y en la query string (`inicio` y `fin`). Si el cliente se desconecta y se reconecta, el servidor no tiene que liberar recursos de sesión ni destruir instancias remotas como ocurriría en sistemas basados en sockets persistentes o RMI. Esto simplifica de manera drástica la tolerancia a fallos del lado del cliente HMI.

B. Diseño de URLs y Semántica de Recursos

El diseño de las URLs se alejó por completo del modelo procedimental (invocación de verbos). En lugar de crear rutas como `/consultarMediciones` o `/configurarUmbral`, se estructuraron sustantivos claros organizados de forma jerárquica:

Diseño Semántico de Endpoints Industriales:

- `/mediciones/MOTOR_01/TEMP` → Representa la colección histórica de datos térmicos del motor.
- `/umbrales/TANQUE_A/HUM1` → Representa la entidad de configuración específica para ese nivel del tanque.

La acción que se desea ejecutar sobre el recurso queda determinada por el método HTTP utilizado: una petición `GET` a la URL de umbrales lee la configuración, mientras que una petición `PUT` a esa misma e idéntica URL modifica los límites físicos.

C. Gestión de Estado e Inyección en Segundo Plano

Un desafío técnico crítico en la migración fue replicar el funcionamiento del `SimuladorSensores` que corría concurrentemente en Java. Dado que HTTP es un protocolo pasivo (el servidor no puede enviar datos al cliente por iniciativa propia si este no los pide primero), el estado de la planta se gestiona de forma centralizada en diccionarios y listas globales de Python (`historial_mediciones`, `historial_alertas`, `umbrales`), los cuales actúan de manera idéntica al modelo clásico de `AlmacenDatos.java`.

Para mantener la inyección continua de datos sin bloquear las peticiones de los operadores en el panel HMI, se utilizó el gestor de eventos de FastAPI (`@app.on_event("startup")`) junto con el bucle asíncrono de eventos de Python (`asyncio.create_task`). Esto levanta un hilo lógico/corutina asíncrona que despierta de forma exacta cada 5 segundos para simular las variaciones físicas del tanque multisensor (`HUM1`, `HUM2`, `HUM3`) y de la temperatura del motor, evaluando en tiempo real si se violan los umbrales configurados para inyectar de forma inmediata los registros en las colecciones de alertas, manteniendo la coherencia total del sistema de monitorización industrial sin comprometer la disponibilidad de la API REST.